

[INFO] BUILD SUCCESS

Observe the logs of Maven command execution where it says *Building jar: /home/osfy/Desktop/spring-petclinic/target/spring-petclinic-2.7.3.jar*.

Now, we have a JAR file ready. The next step is to create a Docker image that contains the JAR file and runtime environment.

So, what is a Dockerfile? The Dockerfile is nothing but the text document that provides build instructions to build the image with your application package.

```
FROM openjdk:11
MAINTAINER osfy
COPY target/spring-petclinic-2.7.3.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Place the Dockerfile in the project root directory. Now, it is time to build a Docker image.

For testing locally with minikube, we should point the local Docker daemon to the minikube internal Docker registry, and build the image to install it in the minikube cluster: *eval \$(minikube docker-env)*.

Next, execute *sudo docker image build -t petclinic:latest*, which is the command from the project root directory.

We will use Docker images available in the local environment and not from the public or private Docker registry; so set the *imagePullPolicy* value to *Never* in the YAML file. Given below is the YAML file for deployment and service, to deploy a sample application in the minikube cluster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: petclinic
  name: petclinic
spec:
  replicas: 1
  selector:
    matchLabels:
      app: petclinic
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: petclinic
    spec:
      containers:
```

```
- image: petclinic:latest
  name: petclinic
  resources: {}
  imagePullPolicy: Never

status: {}
---
apiVersion: v1
kind: Service
metadata:
  name: petclinic
spec:
  type: LoadBalancer
  selector:
    app: petclinic
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

Use the *kubectl apply* command to create service and deployment available in the YAML file.

```
osfy@ubuntu:~/Desktop$ kubectl apply -f petclinic-
deployment.yaml
```

```
deployment.apps/petclinic created
service/petclinic created
```

minikube tunnel runs as a process that helps to create a network route on the host to the service CIDR of the cluster, using the cluster's IP address as a gateway. The *tunnel* command exposes the external IP directly to any program running on the host operating system. Open a new terminal window and execute the *minikube tunnel* command.

```
osfy@ubuntu:~/Desktop$ minikube tunnel
[sudo] password for osfy:
Status:
  machine: minikube
  pid: 94645
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [petclinic]
errors:
  minikube: no errors
  router: no errors
  loadbalancer emulator: no errors
```

Let's verify services:

```
osfy@ubuntu:~/Desktop$ kubectl get services
```